TaskFlow

Project & Task Management System

Project Documentation



TABLE OF CONTENTS

Project Proposal

1. Project Overview

2. Project Objectives

3. Project Scope

TaskFlow – Project Plan

1. Timeline (Gantt Chart)

2. Milestones and Deliverables

3. Resource Allocation

3. Requirements Gathering

Task Assignment & Roles

Risk Assessment and Mitigation Plan

KPIs (Key Performance Indicators)

TaskFlow – Project Proposal

1. Project Overview

- TaskFlow is a project and task management system that helps teams and companies organize their work, assign tasks, and track project progress in a clear and organized way.
- Instead of using different tools to manage tasks, track deadlines, and monitor project progress, TaskFlow brings these functions together in one platform. Users can create projects, assign tasks to team members, monitor task statuses, and track overall project progress. This helps teams organize their work and collaborate more effectively.

2. Project Objectives

1. Organize Work

- Bring projects and tasks together in one platform instead of using multiple tools to manage work.

2. Improve Team Collaboration

- Make it easier for team members to communicate and share updates related to tasks and projects.

3. Manage Time and Deadlines

- Help teams organize project schedules, track task deadlines, and identify dependencies between tasks.

4. Support Decision-Making

- Provide reports and statistics that help users understand project progress, evaluate team performance, and identify the people responsible for each project.

3. Project Scope

A. In Scope

1. Workspace, Team, and Project Management

- Allow users to create workspaces, teams, and projects while organizing users and managing their permissions.

2. Task Management Using Kanban Boards

- Allow users to create tasks, assign them to team members, set priorities and statuses, and organize them into columns such as To Do, In Progress, and Done.

3. Planning and Time Tracking

- Use Gantt charts to display project schedules and show dependencies between tasks. The system will also support tracking the time spent completing tasks.

4. Team Collaboration

- Provide comments, attachments, notifications, and real-time work updates to help team members stay informed about changes and collaborate more effectively.

5. Reports, Analytics, and System Administration

- Provide dashboards for monitoring project progress, an activity log, search and filtering features, task templates, basic automation features, and an admin panel.

B. Out of Scope

The current version of TaskFlow focuses on project and task management, team collaboration, and tracking project progress. Therefore, the following features will not be included in the current version:

1. Payment and Billing Systems

- The system will not include online payment methods, paid subscription management, or customer invoicing. This is because the main purpose of the project is to manage projects and tasks, not financial transactions.

2. Built-in Video Conferencing

- The current version will not support video calls, voice calls, or meetings directly within the platform.

CREATED BY TemplateLAB

Project: Project & Task Management System (Kanban + Gantt)

1. Timeline (Gantt Chart)

Project Milestones			DURATION (Weeks)	WEEK 1	WEEK 2	WEEK 3	WEEK 4
START	**END**	**DESCRIPTION**					
Week 1	**Week 1**	**Milestone 1: Setup, Projects, and Team Management** **Deliverables:** • Complete NTier Architecture implementation • Team and workspace management system • Project creation and customization • Multi-role authentication with granular permissions	**1**	**Milestone 1**			
Week 2	**Week 2**	**Milestone 2: Kanban Boards and Task Management** **Deliverables:** • Interactive Kanban board with drag-and-drop • Comprehensive task management system • Real-time collaboration using SignalR • Comments and attachments system	**1**		**Milestone 2**		
Week 3	**Week 3**	**Milestone 3: Tracking, Reporting, and Notifications** **Deliverables:** • Interactive Gantt chart view • Time tracking system • Analytics dashboard with visual charts • Comprehensive notification system • Activity logging and audit trail	**1**			**Milestone 3**	
Week 4	**Week 4**	**Milestone 4: Advanced Features and Deployment** **Deliverables:** • Advanced search, filtering, and templates • Automation rules engine • Comprehensive admin panel • Polished UI with dark mode and responsive design • Full deployment on Azure with monitoring • Complete documentation including architecture, API reference, setup guide, and user manual	**1**				**Milestone 4**

© templatelab.com

2. Milestones and Deliverables

Milestone	Week	What Will Be Implemented (Tasks)	Deliverables
Setup, Projects, and Team Management	**Week 1**	**NTier Architecture Setup** • TaskFlow.Web (Presentation Layer): interactive user interface • TaskFlow.Services (Business Logic Layer): project and task management logic • TaskFlow.Data (Data Access Layer): data layer with Repository pattern **Database Design** • Workspaces and projects (with settings) • Boards and board columns (To Do, In Progress, Review, Done, etc.) • Tasks and checklist items • Teams and team membership with roles • Comments, file attachments, audit trail, colored labels **Team Management** • Create workspaces and teams • Invite members via email with customizable permissions • Role hierarchy: Owner, Admin, Member, Viewer • Team member activity tracking **Project Setup** • Create projects within workspaces • Customize boards and columns • Pre-built project templates (Agile, Scrum, Basic) • Project settings and access control	• Complete NTier Architecture implementation • Team and workspace management system • Project creation and customization • Multi-role authentication with granular permissions
Kanban Boards and Task Management	**Week 2**	**Kanban Board** • Interactive board with drag-and-drop using Dragula.js • Move tasks between columns with visual feedback • Real-time updates for all team members using SignalR • Column limits (WIP limits) for workflow optimization • Swimlanes for task categorization **Task Management** • Title and rich text description • Due date and time estimates • Priority levels (Low, Medium, High, Critical) • Colored labels for categorization • Assignee selection (single or multiple) • Subtask checklists with progress tracking • Custom fields for additional data • Task operations: edit, delete, archive; copy and move tasks between projects; task templates for recurring work **Real-time Collaboration** • Instant updates when tasks are modified • "Currently viewing" indicators showing active users • Real-time notifications for task changes	• Interactive Kanban board with drag-and-drop • Comprehensive task management system • Real-time collaboration using SignalR • Comments and attachments system

Milestone	Week	What Will Be Implemented (Tasks)	Deliverables
		• Collaborative editing conflict resolution **Comments & Attachments** • Comment threads on tasks • File attachments (images, documents, etc.) • @mention team members for notifications • Rich text formatting in comments	
Tracking, Reporting, and Notifications	**Week 3**	**Gantt Chart View** • Project visualization using Frappe Gantt • Define task dependencies (finish-to-start, etc.) • Drag to adjust dates and durations • Critical path highlighting • Milestone markers **Time Tracking** • Log time spent on each task • Timer functionality for active work • Team productivity reports • Estimated vs. actual time comparison • Timesheet exports **Dashboard & Analytics (Chart.js)** • Tasks by status (pie chart) • Project progress (progress bars) • Team member workload (bar chart) • Overdue tasks highlighting • Velocity charts for sprints • Burndown/burnup charts **Notification System** • In-app notification center • Email notifications (customizable preferences) • Due date reminders • Assignment notifications • Digest emails (daily/weekly summaries) **Activity Log** • Complete audit trail of all actions • Track who changed what and when • Filter by user, task, or action type • Export activity logs	• Interactive Gantt chart view • Time tracking system • Analytics dashboard with visual charts • Comprehensive notification system • Activity logging and audit trail
Advanced Features and Deployment	**Week 4**	**Advanced Features** • Global search across all tasks and projects • Advanced filtering (by status, priority, assignee, date, labels) • Saved filters and custom views • Reusable task templates • Project archiving and restoration	• Advanced search, filtering, and templates • Automation rules engine • Comprehensive admin panel • Polished UI with dark mode and responsive design • Full deployment on Azure with monitoring • Complete documentation including architecture, API reference,

Milestone	Week	What Will Be Implemented (Tasks)	Deliverables
		• Keyboard shortcuts for power users **Automation Rules** • Auto-move task when all subtasks complete • Send notification when due date approaches • Auto-assign based on labels or columns • Custom webhook integrations **Admin Panel** • User and team management • Usage statistics and analytics • System settings and configuration • Subscription and billing management • Data export and backup options **UI/UX Enhancements** • Dark mode toggle • Responsive design for mobile devices • Keyboard shortcuts reference • Customizable dashboard widgets • Accessibility compliance (WCAG 2.1) **Deployment** • Deploy to Azure App Service • Configure Azure SQL Database • Set up Azure SignalR Service • Implement Azure Application Insights for monitoring • Configure CI/CD with GitHub Actions • Load testing and performance optimization	setup guide, and user manual

3. Resource Allocation

3.1 Team Members and Their Tasks

The names are empty on purpose, write your team members names in them.

Role	Week 1	Week 2	Week 3	Week 4
Project Manager / Team Leader (name)	Plan the project, create the solution structure (Web, Services, Data) and give tasks to the team.	Follow the Kanban and task work, review the code of the others.	Check the Gantt chart and the reports with the team, review the notification system.	Write the final documentation: architecture, API reference, setup guide and user manual.
Backend Developer	Database design with EF Core, Repository and Unit of Work, ASP.NET Identity with the 4 roles (Owner, Admin, Member, Viewer), workspaces and teams.	Task logic, State pattern for task states, Command pattern for undo/redo, SignalR hub, comments and attachments.	Time tracking, activity log, in-app and email notifications, digest emails.	Global search and filters, automation rules and webhooks, admin panel.
Frontend Developer	Main layout with Bootstrap 5, pages for workspaces, projects and boards.	Kanban board with Dragula, task details page, WIP limits, swimlanes, comments screen.	Gantt chart with Frappe Gantt, dashboard charts with Chart.js, notification center.	Dark mode, responsive design, keyboard shortcuts, WCAG 2.1, dashboard widgets.
QA / DevOps Engineer	Create the GitHub repository, test login and permissions for each role.	Test drag and drop and the real-time updates with more than one user.	Test time tracking, charts and the email notifications.	Azure App Service, Azure SQL Database, Azure SignalR Service, Application Insights, GitHub Actions CI/CD and load testing.

3.2 Technical Resources

Type	Resource	Used for
Framework	ASP.NET Core MVC	The web application (presentation layer)
Data	Entity Framework Core	Database access in the data layer
Security	ASP.NET Identity	Login, roles and permissions
Real-time	SignalR	Live task updates, "currently viewing" indicators, notifications
Front-end	Dragula.js	Drag and drop on the Kanban board
Front-end	Frappe Gantt	Gantt chart view
Front-end	Chart.js	Dashboard charts (pie, bar, burndown, velocity)
Front-end	Bootstrap 5	Responsive design
Cloud	Microsoft Azure (App Service, SQL Database, SignalR Service, Application Insights)	Hosting, database, real-time service and monitoring
Source control	GitHub and GitHub Actions	Code repository and CI/CD

3.3 Design Patterns and Practices

Design Pattern / Practice	Purpose	Where we use it
NTier Architecture	Layer separation for easy maintenance	TaskFlow.Web, TaskFlow.Services and TaskFlow.Data projects

Design Pattern / Practice	Purpose	Where we use it
Repository Pattern	Abstract data operations	Data layer, one repository for each main table
Unit of Work Pattern	Ensure data integrity across transactions	Saving changes to several tables together in one transaction
State Pattern	For managing different task states and transitions	Task states and moving a task between columns
Command Pattern	For undo/redo operations	Undo and redo of task actions
Dependency Injection	For improved architecture and testing	Controllers, services and repositories get their dependencies by injection

2- Literature Review & Technical Foundation

2.1 Literature Review (Similar Systems)

We reviewed three real project management systems that are widely used by teams. The table compares what each system does well, where it is weak, and what TaskFlow will take or improve.

System	Key Strength	Key Weakness	What We Take / Improve
Trello	Very simple Kanban boards with easy drag and drop that any team can learn in minutes.	Limited built-in planning: Gantt-style views, task dependencies and advanced reporting are weak or restricted to paid plans.	We take the simple, clean board experience and add a Gantt chart with task dependencies in the same system.
Jira	Powerful customizable workflows, Scrum and Kanban boards, sprint reports such as burndown, and detailed permissions.	Complex to set up and hard for beginners; designed mainly for software teams.	We take the idea of workflows and burndown charts, and keep the interface simple enough for any kind of team.
Asana	Timeline view, list and board views, reporting dashboards, and rules to automate repeated work.	Advanced timeline, reporting and automation features are mostly limited to paid plans.	We take the idea of automation rules (for example, notify when a due date is close) and include it as a core feature.

TaskFlow combines the simple boards of Trello, the workflow ideas of Jira, and the automation of Asana in one platform that is easy to use.

2.2 System Architecture & Design Patterns

Design Pattern / Practice	Purpose	Where We Use It	Why
N-Tier Architecture	Layer separation for easy maintenance.	TaskFlow.Web (presentation), TaskFlow.Services (business logic) and TaskFlow.Data (data access) projects.	Each team member can work on one layer without breaking the others, and each layer is easier to test.
Repository Pattern	Abstract data operations.	Data layer, one repository for each main table (Projects, Tasks, Comments, ...).	Hides EF Core details from the Services layer and makes unit testing easier.
Unit of Work Pattern	Ensure data integrity across transactions.	Saving changes to several tables together in one transaction (for example, moving a task and writing the activity log).	Prevents partial saves and data loss if one operation fails.
State Pattern	Manage different task states and transitions.	Task states and moving a task between Kanban columns.	Each state has its own rules, so we avoid long if/else blocks and invalid transitions.
Command Pattern	Support undo and redo operations.	Undo and redo of task actions (edit, move, delete).	Each action is stored as an object, so it can be reversed or replayed.
Dependency Injection	Improve architecture and testing.	Controllers, services and repositories receive their dependencies by injection.	Reduces coupling between classes and allows mock objects in tests.

2.3 Technology Stack

Technology	Version	Used For	Why We Chose It
ASP.NET Core MVC	10 (.NET 10, LTS)	The web application (presentation layer).	Matches the training track, supports N-Tier, and .NET 10 is supported until November 2028 (.NET 8 and 9 end on 10 Nov 2026).
Entity Framework Core	10.x	Database access in the data layer.	Code-first migrations and LINQ queries, and it works directly with the Repository pattern.
SQL Server / Azure SQL Database	2022	Main relational database.	Data is relational (Workspaces, Projects, Tasks, Users) and needs transactions.
ASP.NET Identity	Included with .NET 10	Login, roles and permissions.	Ready and tested support for the four roles: Owner, Admin, Member and Viewer.
SignalR	Included with .NET 10	Live task updates, "currently viewing" indicators and notifications.	Real-time updates to all board users within 1 to 2 seconds.
Dragula.js	3.7.3	Drag and drop on the Kanban board.	Lightweight library with no dependencies.
Frappe Gantt	1.x	Gantt chart view.	Open-source and supports task dependencies and drag to change dates.
Chart.js	4.x	Dashboard charts (pie, bar, burndown, velocity).	Simple API and responsive charts.
Bootstrap	5.3	Responsive design.	Fast responsive layout and ready components, with dark mode support.
Microsoft Azure	App Service, Azure SQL, SignalR Service, Application Insights	Hosting, database, real-time service and monitoring.	One cloud platform that works well with .NET and gives built-in monitoring.
GitHub and GitHub Actions	Cloud service	Code repository and CI/CD.	Team collaboration with pull requests, and automatic build and deploy to Azure.

Self-check — Task 2:

- ☑ Are 3 similar systems compared in a table?
- ☑ Is every design pattern linked to a place in our project?
- ☑ Does every technology have a version and a reason?
- ☑ Are there no cut-off lines or missing paragraphs?

3. Requirements Gathering

3.1 Stakeholder Analysis

TaskFlow stakeholder analysis is concerned with identifying the key stakeholders of the system and their requirements. Each stakeholder uses the system in accordance with their roles and authorizations.

Stakeholders and Responsibilities

Stakeholder	Needs and Responsibilities
Workspace Owner	Creates workspaces, invites members, manages teams, and controls access permissions.
Administrator	Manages users, roles, permissions, and system settings.
Project Manager	Creates projects, organizes tasks, assigns team members, manages deadlines, and monitors project progress.
Team Member	Views assigned tasks, updates task status, creates subtasks, tracks working hours, adds comments, and uploads attachments.
Viewer	Views projects, tasks, boards, and reports without modifying project data.
System Administrator	Monitors system performance, manages backups, and reviews system activity and security logs.

Stakeholder Needs

The major requirements for TaskFlow include:

- **Project Organization:** Manage workspace, project, task, and sub-task organization in one solution.
- **Team Collaboration:** Promote communication using comments, mentions, and file attachment features.
- **Workflow Management:** Manage task progress using Kanban and Gantt diagrams.
- **Real-time Updates:** Alert members of your team about changes in tasks or projects.
- **Reporting and Analytics:** Track project progress, late tasks, and staff workload.
- **Security:** Keep project data secure with authentication and roles.
- **Automation:** Avoid repetition through automated reminders and workflow automation rules.

3.2 User Stories

User stories provide description of system's capabilities from user point of view.

ID	User Story	Priority
US-01	As a workspace owner, I want to create workspaces and invite members so that I can organize team collaboration.	High
US-02	As an administrator, I want to manage user roles and permissions so that access is controlled.	High
US-03	As a project manager, I want to create projects and customize boards so that I can organize project activities.	High

ID	User Story	Priority
US-04	As a project manager, I want to create tasks and assign them to team members so that responsibilities are clear.	High
US-05	As a team member, I want to move tasks between Kanban columns so that I can update their progress easily.	High
US-06	As a team member, I want to create subtasks and track their completion so that I can divide large tasks into smaller activities.	High
US-07	As a team member, I want to add comments and attachments so that I can collaborate with other members.	High
US-08	As a project manager, I want to view Gantt charts so that I can manage deadlines and task dependencies.	Medium
US-09	As a team member, I want to record time spent on tasks so that I can compare estimated and actual working hours.	Medium
US-10	As a project manager, I want to view reports and dashboards so that I can monitor project performance.	High
US-11	As a team member, I want to receive notifications about assignments and deadlines so that I do not miss important updates.	High
US-12	As a user, I want to search and filter tasks so that I can find information quickly.	Medium
US-13	As a project manager, I want to configure automation rules so that repetitive actions can be performed automatically.	Medium
US-14	As an administrator, I want to review activity logs so that I can track changes and identify who performed each action.	Medium

3.3 Use Cases

Use case scenarios explain how the users use TaskFlow for their particular needs.

Main Use Cases

Use Case 1: Manage Workspace and Team

Actor	Workspace Owner
Prerequisite	The user has signed in.
Main Flow	1. The owner designs an office. 2. The owner calls the team members. 3. Owner allocates functions and rights. 4. This system archives the workspace and membership details. 5. Invitations are then sent to the members through the system.
Postcondition	Workspace creation is completed, and members can have access to it based on their permissions.

Use Case 2: Create Project and Tasks

Actor	Project Manager
Prerequisite	The user has the ability to log in and can create projects.
Main Flow	1. The manager designs the project. 2. The manager chooses the project template or makes one from scratch. 3. The manager sets the assignments along with their priorities and deadlines. 4. The manager delegates assignments to team members. 5. The application saves the project and presents the tasks within the board.
Postcondition	The project and tasks are accessible by authorized users.

Use Case 3: Update Task Status

Actor	Team Member
Prerequisite	The member is able to see the project board.
Main Flow	1. The user starts the Kanban board. 2. The member transfers the task into a different column. 3. The system verifies the new status. 4. The system stores the status of the task. 5. Other participants on the board are updated by SignalR.
Postcondition	Task status update and activity log have been accomplished.

Use Case 4: Collaborate on Tasks

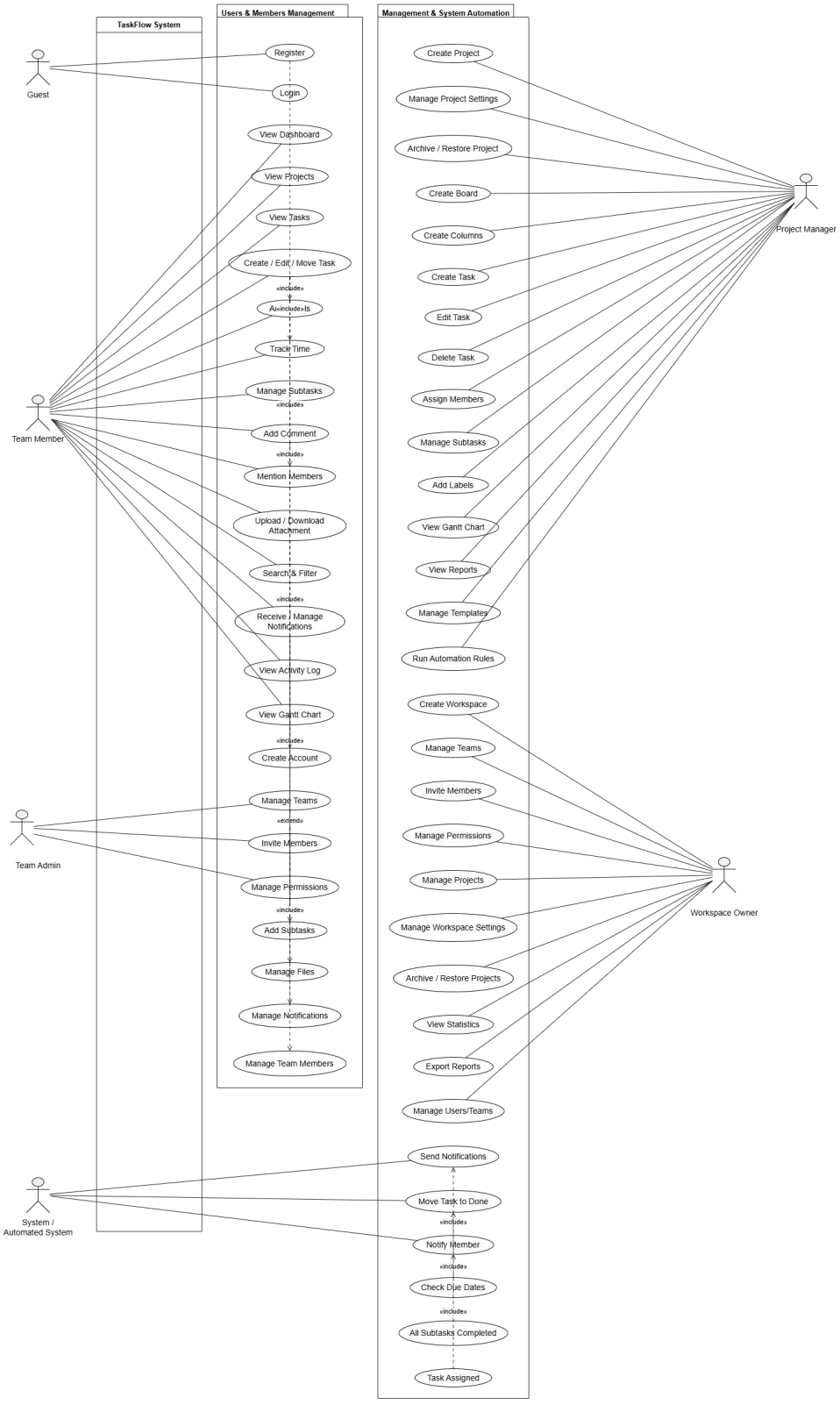
Actor	Team Member
Prerequisite	The member is authorized to work on the task.
Main Flow	1. The member will access task information. 2. The participant makes a comment or refers to another participant. 3. The member can upload the attachment when necessary. 4. This system stores the data and provides necessary notifications.
Postcondition	The task discussion and attachments are accessible by the authorized users.

Use Case 5: Monitor Project Progress

Actor	Project Manager
Prerequisite	The manager is authorized to access the project reports.
Main Flow	1. The manager accesses the dashboard. 2. The system shows task statuses, progress on projects, late tasks, and team workloads. 3. The manager begins to analyze the Gantt chart to study the schedule. 4. The manager uses this data to pinpoint any problems in the schedule.
Postcondition	Manager will be able to assess the status of projects.

Use Case Diagram

The use case diagram shows the interaction between actors in the TaskFlow and the major functionalities of the system.



3.4 Functional Requirements

Functional requirements are used to describe the functionality that TaskFlow system must offer.

ID	Functional Requirement
FR-01	The system shall allow users to register, log in, and log out securely.
FR-02	The system shall allow workspace owners to create and manage workspaces.
FR-03	The system shall allow authorized users to create teams and invite members.
FR-04	The system shall support role-based access control for owners, administrators, members, and viewers.
FR-05	The system shall allow project managers to create, edit, archive, and restore projects.
FR-06	The system shall provide customizable Kanban boards and columns.
FR-07	The system shall allow users to create, assign, update, delete, and move tasks according to their permissions.
FR-08	The system shall support subtasks, priorities, labels, deadlines, and task dependencies.
FR-09	The system shall allow users to add comments, mention team members, and upload attachments.
FR-10	The system shall update task changes in real time for connected users.
FR-11	The system shall provide Gantt charts to visualize schedules, milestones, and dependencies.
FR-12	The system shall allow users to record working time and compare estimated and actual time.
FR-13	The system shall provide dashboards and reports showing project progress and team workload.
FR-14	The system shall send notifications for assignments, comments, and approaching deadlines.
FR-15	The system shall maintain activity logs for important user actions and changes.
FR-16	The system shall provide search, filtering, and reusable task templates.
FR-17	The system shall support configurable automation rules for repetitive workflow actions.
FR-18	The system shall allow users to customize the interface, including dark mode.
FR-19	The system shall provide administrative tools for user management, system settings, and data export.

3.5 Non-Functional Requirements

Non-functional requirements state the quality features and limitations of TaskFlow.

ID	Category	Requirement
NFR-01	Performance	The system should load standard pages within 3 seconds under normal operating conditions.
NFR-02	Performance	Task updates should be reflected to connected users within 2 seconds under normal network conditions.
NFR-03	Security	The system shall use secure authentication and role-based authorization to protect user and project data.
NFR-04	Security	Passwords shall be stored using secure password hashing, and sensitive data shall be protected during transmission.
NFR-05	Usability	The interface shall be clear and easy to use, with consistent navigation and understandable error messages.
NFR-06	Responsiveness	The interface shall adapt to desktop, tablet, and mobile screen sizes.
NFR-07	Reliability	The system shall validate user inputs and handle errors without losing previously saved data.
NFR-08	Availability	The deployed system should remain available during normal operating periods, excluding planned maintenance.
NFR-09	Maintainability	The system shall use N-Tier Architecture and separation of concerns to simplify maintenance and testing.

ID	Category	Requirement
NFR-10	Scalability	The system architecture should support increasing numbers of users, projects, and tasks.
NFR-11	Data Integrity	The system shall maintain consistent relationships between projects, tasks, users, and other database entities.
NFR-12	Compatibility	The system shall support modern web browsers such as Microsoft Edge, Google Chrome, and Mozilla Firefox.
NFR-13	Accessibility	The interface should follow WCAG 2.1 accessibility guidelines where practical.
NFR-14	Backup and Recovery	The system shall support database backups and data recovery procedures.

3.6 Summary

The requirements collection stage sets the stakeholders, user stories, use cases, functional requirements, and non-functional requirements of TaskFlow. The requirements set the basis for architecture design, database design, user interface design, and testing.

Task Assignment & Roles

1. Teamwork Approach

- We decided that all team members will work together on developing the TaskFlow project. This is to ensure that every team member applies what they learned during the training period and understands every part of the project, including its components and how it works.
- The collaboration will cover all stages of development: User Interface and User Experience design (UI/UX), Frontend and Backend development, database design, API development and integration, and finally, system testing to ensure that all parts work together correctly.

2. Task and Responsibility Distribution

- The programming tasks will be divided when the actual project planning phase begins.
- The distribution will depend on the project requirements and the type of tasks needed, with agreement among all team members.

3. Communication and Progress Tracking

- All team members will continuously follow the project's progress through discussions and sharing updates.
- This will help every member stay informed about the parts being developed and allow the team to work together to solve problems and understand how the different parts of the project work together.

4. Goal of This Approach

- The goal is to ensure that all team members understand the project as a whole, not just the parts they are responsible for.
- It also aims to improve collaboration among team members.

Risk Assessment and Mitigation Plan

Probability and Impact are our own estimate: Low, Medium or High.

No.	Risk	Probability	Impact	Mitigation Plan
1	Not enough time to finish the project	Medium	High	Make a weekly plan from the Gantt chart, start with the most important features first, check the progress every few days, and make the less important features simpler if we are late.
2	Data loss	Low	High	Use the Unit of Work pattern so changes are saved together in one transaction, keep backups of the Azure SQL database, keep the code on GitHub, and test delete and archive before using them.
3	Edit conflicts (two users change the same task)	Medium	Medium	Add the conflict resolution for collaborative editing, show who is viewing the task with SignalR, and show a warning before one change overwrites another.
4	A team member does not know how to do a task or finds it hard	Medium	Medium	Ask the instructor, or all the team help him together. We also share what we learn in the team meetings and use the documentation of ASP.NET Core, EF Core and SignalR.
5	Wrong design of the front-end or the database	Medium	High	Read the project requirements carefully from the beginning and analyze them well before coding. Design the database and the screens first, then review them with the team and the instructor.
6	A team member is late in delivering a task or has personal circumstances	Medium	Medium	Set a clear deadline for each task, follow up in short meetings, and move the task to another member if needed. Each part has a second person who knows it.
7	Real-time features with SignalR are hard to implement	Medium	High	Start early in Week 2 with a simple version, then improve it step by step, and test it with more than one user.
8	Security and permission problems with the roles	Low	High	Use ASP.NET Identity, test each role (Owner, Admin, Member, Viewer) separately, and check the permissions in the server side, not only in the screens.
9	Slow performance when there are many users or lots of data	Medium	Medium	Do the load testing and performance optimization in Week 4, use paging, and write simple efficient queries.
10	Problems when deploying to Azure	Medium	High	Try a first deployment early, before Week 4, set up CI/CD with GitHub Actions, and check Application Insights if something fails.

KPIs (Key Performance Indicators)

Targets are our suggestion and can be changed after we test the system.

No.	KPI	What it means	Target / How we measure it
1	Response time	The system is fast and responds without big delay.	Pages, the Kanban board and the main actions respond in about 2 seconds or less (checked in the load testing in Week 4).
2	Correct functions	The main functions work correctly and without errors.	All main features pass our testing and there are no critical bugs when we deliver.
3	System integration	The Frontend, the Backend and the API work together correctly.	Every screen gets and saves its data through the API correctly, and the integration tests pass.
4	Ease of use	The user can deal with the system easily.	A new user can create a project, add a task and move it on the Kanban board without any help.
5	On-time delivery	The team finishes the agreed features on the planned time.	All 4 milestones are finished at the end of their weeks, as in the Gantt chart.
6	System uptime	The system is available and working most of the time after deployment.	About 99% uptime on Azure, checked with Application Insights.
7	Real-time update speed	When someone changes a task, the other users see the change quickly.	The change appears for the other users in about 1 to 2 seconds using SignalR.
8	Security and permissions	Every role can do only what it is allowed to do.	All the tests for the roles (Owner, Admin, Member, Viewer) pass.
9	User adoption rate	The users really use TaskFlow to manage their work.	In the final test, every team member uses it for his own tasks and projects.
10	Successful deployment	The project is online and updates automatically.	Deployed on Azure App Service with Azure SQL, and the GitHub Actions CI/CD runs without errors.